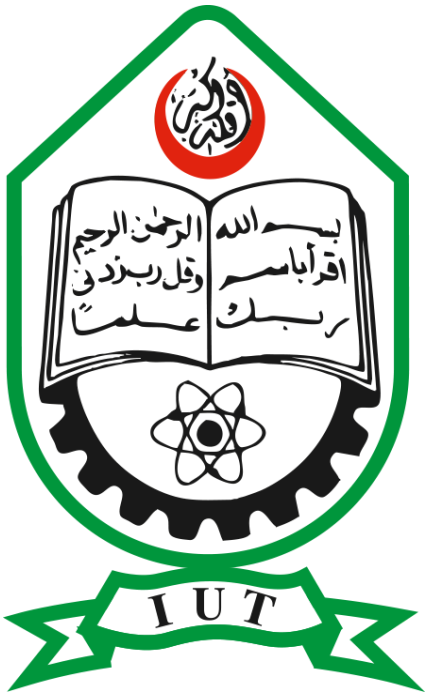


ISLAMIC UNIVERSITY OF TECHNOLOGY

PROJECT REPORT

Stock Market Crash Simulator

Using Markov Chains and Geometric Brownian Motion for Regime-Switching Price Simulation and Statistical Crash Detection

Submitted by

TEAM 1xBet

Shaheen Rahman Dibya (230021315)

&

Ishrak Samin (230021303)

EEE 4408 — Group Project

Abstract

This report presents a MATLAB simulator that models stock market behavior and statistically detects crashes in real time.

Key System Components

1. Three-State Markov Chain: Governs market regimes (Bull, Bear, and Crash).
2. Geometric Brownian Motion (GBM): Generates realistic daily price paths with drift and volatility parameters that adapt to the active regime This is used for generating the everyday new price of the stock .
3. Binary Hypothesis Test: Monitors simulated returns to detect statistically elevated risk.This is used to detect crashes

The system is delivered as an interactive dashboard featuring live parameter adjustment, real-time risk visualization, and run-history logging. The report covers the project's motivation, theory, design, implementation, and a representative simulation, concluding with its educational benefits, technical advantages, and future improvements.

Contents

Abstract

1. Introduction and Motivation
2. Project Objectives
3. Theoretical Background
   - 3.1 Markov Chains for Modeling Market Regimes
   - 3.2 Geometric Brownian Motion for Price Evolution
   - 3.3 Statistical Hypothesis Testing for Crash Detection
4. System Design: A Regime-Switching Price Engine
5. Implementation: The Interactive Dashboard
6. Results and Discussion
7. Benefits of the Project
8. Advantages of Our Approach
9. Limitations and Recommended Improvements
10. Conclusion

1. Introduction

Real stock markets do not function as a smooth, constant-volatility random process. Historical episodes the 1929 crash, 2010-11 Bangladesh Market crash,and the 2020 COVID-19 crash share a common signature: prices fall sharply while volatility spikes at the same time, rather than declining gradually. A model that assumes constant volatility throughout therefore misses exactly the kind of risk build-up that tends to precede a real crash.

This observation motivates two related needs, first, a realistic model of price behaviour should allow volatility and drift to change over time, reflecting the market's underlying mood - growing steadily, correcting moderately, or panicking outright. Second, because reliably predicting the exact timing of a future crash is a well-known unsolved problem in finance - if it were reliably solvable, market participants would act on the prediction and the market would immediately reprice around it - a more realistic and achievable goal is to statistically detect that a crash is already, currently, underway, using only the price data that is actually observable.

This project was undertaken with the explicit requirement of building on concepts from EEE 4407 (Random Signal and Processes).It brings together three concepts which are as follows:

Markov chains, stochastic processes based on Brownian motion, and statistical hypothesis testing into a single, coherent, interactive system, rather than treating them as separate, isolated exercises.

2. Project Objectives

The project was scoped around four objectives:

1. **Simulate realistic market regimes:** This is done by modelling the market as a three-state Markov chain - Bull, Bear, and Crash each with its own characteristic behaviour.
2. **Generate a realistic daily price path:** This is done by using Geometric Brownian Motion, with drift and volatility parameters that depend on the currently active regime.
3. **Detect crashes statistically:** This is done by using a rolling window binary hypothesis test on the simulated returns, rather than relying on visual inspection of a price chart.
4. **Visualise the entire system live:** This is done through an interactive MATLAB dashboard that lets a user adjust parameters and observe the effect immediately, instead of a script that must be edited and re-run for every change.

3. Theoretical Background

3.1 Markov Chains for Modeling Market Regimes

A Markov chain is a stochastic process in which the probability of transitioning to the next state depends only on the current state, and not on the sequence of states that preceded it This project defines three market regimes which are Bull, Bear, and Crash, Models the day-to-day transitions between them using the fixed transition probability matrix shown below in Table 1.

From \ To	BULL	BEAR	CRASH
BULL	85%	14%	1%
BEAR	20%	75%	5%
CRASH	40%	30%	30%

Table 1. Transition probability matrix used in the simulation.

Each row describes the probabilities of moving from that regime to each of the three regimes the following day, and therefore sums to one. The chosen values are illustrative but realistic: Bull markets are persistent (85% probability of staying in the Bull state the next day), a Crash is rare to enter directly from a Bull market (1%) but is highly volatile once active. It has 30% probability of persisting for another day and 40% probability of reverting straight back to a Bull market.

3.2 Geometric Brownian Motion for Price Evolution

To generate the simulated price itself, the project uses Geometric Brownian Motion (GBM). GBM is used instead of an ordinary, additive random walk applied directly to price because an ordinary random walk can become a value below 0 which is meaningless for a real stock price, as stock prices can never ever be below 0. GBM instead applies the random walk to the logarithm of price; because the exponential function is always positive, the simulated price can never become (-)ve.

The discrete-time update rule used in this project is:

$$S(t) = S(t-1) \cdot \exp[(\mu - \tfrac{1}{2}\sigma^2)\Delta t + \sigma\sqrt{\Delta t} \cdot Z]$$

where S(t) is the simulated price on day t, μ is the drift, σ is the volatility(variance), Δt is the length of one trading day (1/252 of a year), and Z is a freshly drawn standard normal random variable MATLAB's randn generated independently every day. The √Δt scaling reflects a fundamental property of random walks: because the variance of a sum of independent random increments grows linearly with the number of steps, the standard deviation, and therefore the typical spread of the walk, grows with the square root of elapsed time. The $-\frac{1}{2}\sigma^2$ term is a small but necessary correction sometimes called the Ito correction. It accounts for the fact that a percentage increase followed by an equal percentage decrease does not return a price to its starting value. Without this, the average simulated price path would drift systematically away from the intended trend.

Crucially, mu and sigma are not constant throughout the simulation. Instead, they are from the currently active Markov regime at every time step.

3.3 Statistical Hypothesis Testing for Crash Detection

The third technique determines, from the simulated price data alone, whether a crash is statistically likely to be underway. This is framed as a classical binary hypothesis test:

H_0 (null hypothesis): the market is behaving normally; no alarm.

H_1 (alternative hypothesis): a crash is statistically underway.

At every trading day, using a 10-day sliding window of the most recent returns, the system evaluates whether the available evidence supports rejecting H_0 in favour of H_1. In this implementation, the alarm is raised when the rolling sample variance of the last 10 days' returns exceeds a fixed threshold (0.0015), when the price has fallen more than 15% from its most recent peak within that same window, or when the underlying simulated regime has itself switched to Crash.

The ten-day window size represents a deliberate trade-off: a much shorter window would produce an extremely noisy variance estimate, while a much longer window would dilute a genuine, sudden regime shift among many unrelated, calmer days, delaying detection. A rolling window keeps the test sensitive to current conditions rather than the entire historical record.

It is worth noting explicitly that the third condition — checking the simulated regime directly — is only possible because this is a simulation in which the “true” state is known by construction. A genuinely blind detector, operating on real, unlabelled market data, could rely only on the first two, directly observable

conditions: variance and drawdown. This point, and how it can be addressed, is discussed further in Section 9.

4. System Design: A Regime-Switching Price Engine

The overall system operates as a single integrated pipeline, executed once per simulated trading day:

- **Step 1 — Draw the regime:** the next day's market regime is drawn from the Markov transition matrix (Section 3.1), based on the current regime.
- **Step 2 — Look up parameters:** the drift and volatility appropriate to that regime are looked up according to Table 2.
- **Step 3 — Step the price forward:** those regime-specific values of μ and σ are substituted into the GBM update rule (Section 3.2) to advance the simulated price by one day.

Regime	Drift (μ)	Volatility (σ)
Bull	μ_{bull} (user-set, e.g. 8%)	σ_{bull} (user-set, e.g. 12%)
Bear	$\min(-5\%,\ \mu_{bull} - 10\%)$	$\max(1.5 \times \sigma_{bull},\ 25\%)$
Crash	−50% (fixed)	$\max(2.5 \times \sigma_{bull},\ 50\%)$

Table 2. Regime-dependent drift and volatility parameters used by the price engine.

This design means that Bear and Crash regimes do not merely nudge the price path slightly — they impose a materially different statistical “personality” on that portion of the simulation, with heavier volatility and, during a Crash, a strongly negative expected trend. Because the price path and the hidden regime sequence are generated together, the system can subsequently be used to check how well the hypothesis test described in Section 3.3 actually recovers the hidden regime from the price data alone — a genuine advantage of working with a fully simulated system, discussed further in Section 8.

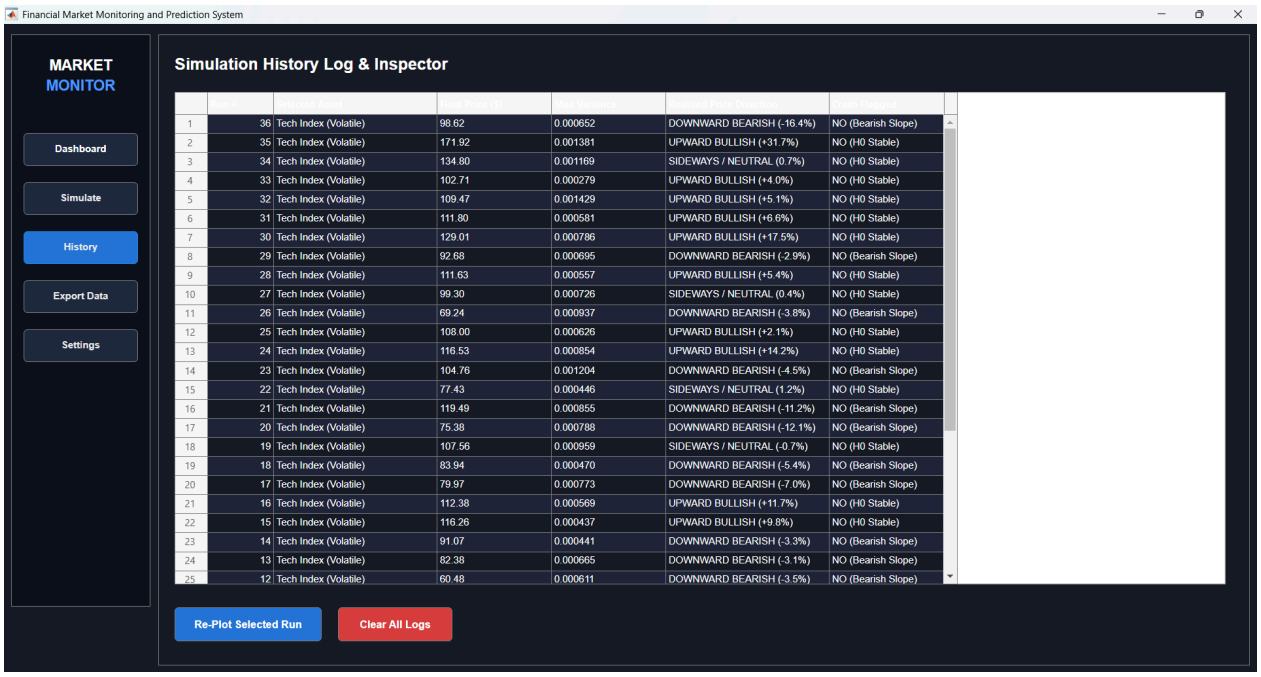
5. Implementation: The Interactive Dashboard

The system was implemented as a self-contained MATLAB App Designer application, structured as three navigable views rather than a single static script.

- **Dashboard view:** Displays the live simulated price chart with detected crash points in red circles, a semicircular risk gauge (0–100%, colour-coded green to amber to red as risk increases), and status bar showing the currently active model parameters, the current hypothesis-test outcome (H_0/H_1), and a short-window trend indicator, labelled "Bullish" or "Bearish" based on whether the 10-day realised return exceeds +1.5% or falls below −1.5%.



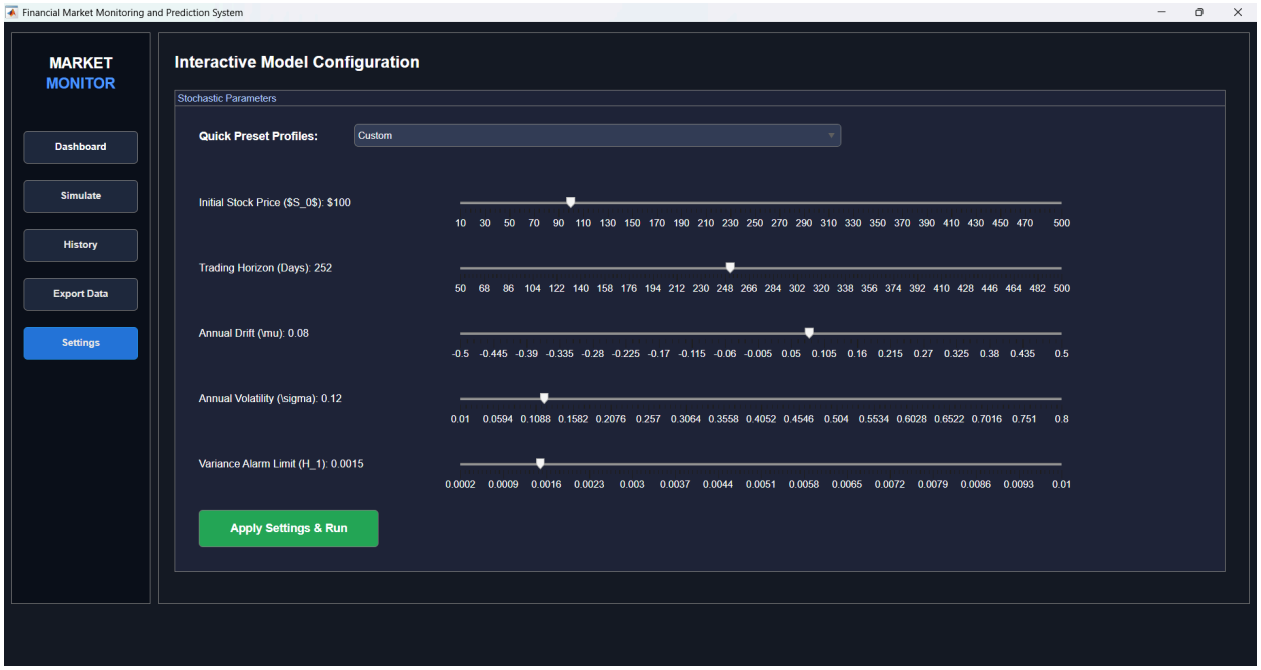
- **History view:** This logs every simulation run in a table (asset preset, final price, peak variance, and whether a crash was flagged), with the ability to re-plot any past run or clear the log entirely.



1	36	Tech Index (Volatile)	98.62	0.000652	DOWNWARD BEARISH (-16.4%)	NO (Bearish Slope)
2	35	Tech Index (Volatile)	171.92	0.001381	UPWARD BULLISH (+31.7%)	NO (H0 Stable)
3	34	Tech Index (Volatile)	134.80	0.001169	SIDEWAYS / NEUTRAL (0.7%)	NO (H0 Stable)
4	33	Tech Index (Volatile)	102.71	0.000279	UPWARD BULLISH (+4.0%)	NO (H0 Stable)
5	32	Tech Index (Volatile)	109.47	0.001429	UPWARD BULLISH (+5.1%)	NO (H0 Stable)
6	31	Tech Index (Volatile)	111.80	0.000581	UPWARD BULLISH (+6.6%)	NO (H0 Stable)
7	30	Tech Index (Volatile)	129.01	0.000786	UPWARD BULLISH (+17.5%)	NO (H0 Stable)
8	29	Tech Index (Volatile)	92.68	0.000695	DOWNWARD BEARISH (-2.9%)	NO (Bearish Slope)
9	28	Tech Index (Volatile)	111.63	0.000557	UPWARD BULLISH (+5.4%)	NO (H0 Stable)
10	27	Tech Index (Volatile)	99.30	0.000726	SIDEWAYS / NEUTRAL (0.4%)	NO (H0 Stable)
11	26	Tech Index (Volatile)	69.24	0.000937	DOWNWARD BEARISH (-3.8%)	NO (Bearish Slope)
12	25	Tech Index (Volatile)	108.00	0.000626	UPWARD BULLISH (+2.1%)	NO (H0 Stable)
13	24	Tech Index (Volatile)	116.53	0.000854	UPWARD BULLISH (+14.2%)	NO (H0 Stable)
14	23	Tech Index (Volatile)	104.76	0.001204	DOWNWARD BEARISH (-4.5%)	NO (Bearish Slope)
15	22	Tech Index (Volatile)	77.43	0.000446	SIDEWAYS / NEUTRAL (1.2%)	NO (H0 Stable)
16	21	Tech Index (Volatile)	119.49	0.000855	DOWNWARD BEARISH (-11.2%)	NO (Bearish Slope)
17	20	Tech Index (Volatile)	75.38	0.000788	DOWNWARD BEARISH (-12.1%)	NO (Bearish Slope)
18	19	Tech Index (Volatile)	107.56	0.000959	SIDEWAYS / NEUTRAL (-0.7%)	NO (H0 Stable)
19	18	Tech Index (Volatile)	83.94	0.000470	DOWNWARD BEARISH (-5.4%)	NO (Bearish Slope)
20	17	Tech Index (Volatile)	79.97	0.000773	DOWNWARD BEARISH (-7.0%)	NO (Bearish Slope)
21	16	Tech Index (Volatile)	112.38	0.000569	UPWARD BULLISH (+11.7%)	NO (H0 Stable)
22	15	Tech Index (Volatile)	116.26	0.000437	UPWARD BULLISH (+9.8%)	NO (H0 Stable)
23	14	Tech Index (Volatile)	91.07	0.000441	DOWNWARD BEARISH (-3.3%)	NO (Bearish Slope)
24	13	Tech Index (Volatile)	82.38	0.000665	DOWNWARD BEARISH (-3.1%)	NO (Bearish Slope)
25	12	Tech Index (Volatile)	60.48	0.000611	DOWNWARD BEARISH (-3.5%)	NO (Bearish Slope)

- **Settings view** — provides five live sliders (initial price, simulation horizon, drift, volatility, and alarm threshold) whose on-screen labels update as they are dragged, together with three quick preset

profiles: a High-Volatility profile representing high-growth, high-risk assets; a Low-Volatility profile representing “safe-haven” assets such as government bonds or blue-chip dividend stocks; and a High-Crash-Risk profile designed specifically to stress-test the crash detector under conditions resembling a severe financial crisis or panic-selling event.

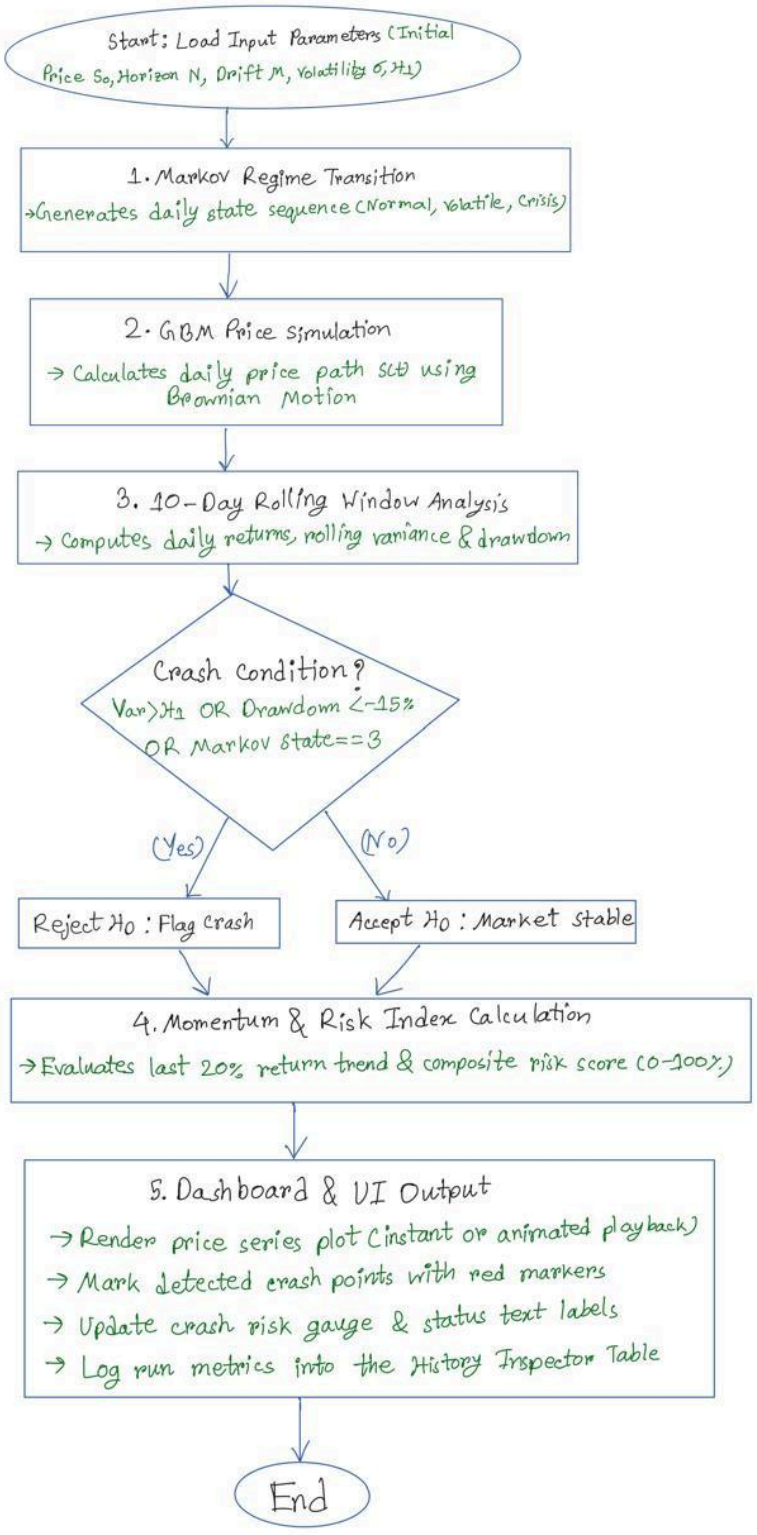


Additional features include an optional animated (“live playback”) mode that draws the price path incrementally rather than plotting it instantly, and one-click export of any run's price and variance series to CSV or Excel for further analysis outside MATLAB.

Presenting the project as an interactive application, rather than a script that must be edited and re-run for every parameter change, was a deliberate design choice intended to make the underlying stochastic concepts more tangible and easier to explore — both for the group during development, and for an audience during a live demonstration.

6. How the code works:

The Actual code is more than 300 lines long and most of the portion of it is how we actually implemented the UI. The main heart of the code however is the part where we used the Markov chain, Geometric Brownian Motion and Hypothesis testing to simulate the project. A brief flow chart has been given below for the code along with the main code:


Start: Load Input Parameters (Initial Price S_0, Horizon N, Drift μ, Volatility σ, H_1)
1. Markov Regime Transition
→Generates daily state sequence (Normal, Volatile, Crisis)
2. GBM Price Simulation
→ Calculates daily price path S(t) using Brownian Motion
3. 10-Day Rolling Window Analysis
→ Computes daily returns, rolling variance & drawdown
Crash Condition?
Var > H_1 OR Drawdown < -15%
OR Markov State == 3
(Yes)
(No)
Reject H_0 : Flag Crash
Accept H_0 : Market Stable
4. Momentum & Risk Index Calculation
→ Evaluates last 20% return trend & composite risk score (0-100%)
5. Dashboard & UI Output
→ Render price series plot (instant or animated playback)
→ Mark detected crash points with red markers
→ Update crash risk gauge & status text labels
→ Log run metrics into the History Inspector Table
End

```
function runSimulation()
    run_count = run_count + 1;
    params.animate = cb_anim.Value;
    N = round(sld_N.Value);
    S0 = sld_S0.Value;
    dt = 1/252;

    current_mu = sld_mu.Value;
    current_sig = sld_sig.Value;

    % 3-State Transition Matrix
    P = [0.85, 0.14, 0.01;
         0.20, 0.75, 0.05;
         0.40, 0.30, 0.30];

    % Markov Chain State Generation
    states = zeros(1, N); states(1) = 1;
    for t = 2:N
        r = rand();
        cum_prob = cumsum(P(states(t-1), :));
        states(t) = find(r <= cum_prob, 1, 'first');
    end

    % Drift & Volatility Vector per State
    mu_vec = [current_mu, min(-0.05, current_mu - 0.10), -0.50] * dt;
    sigma_vec = [current_sig, max(current_sig * 1.5, 0.25), max(current_sig * 2.5, 0.50)] * sqrt(dt);

    prices = zeros(1, N); prices(1) = S0;
    dW = randn(1, N);

    for t = 2:N
        st = states(t);
        prices(t) = prices(t-1) * exp((mu_vec(st) - 0.5*sigma_vec(st)^2) + sigma_vec(st)*dW(t));
    end
```

Here the variable P is taken as the transition matrix of the markov chain. It signifies that if we are in bull state, there is an 85 percent probability that we will stay in bull, 14 percent we are going to demote to bear and a 1 percent chance that the market is going to crash.
The markov chain generation generates random number for probability which ultimately dictates the state where the market is going to go.
Finally we have the geometric brownian motion which generates the random price of product or stock for each new day.

```
% Hypothesis Testing for Crash Detection
window_size = 10;
var_est = zeros(1, N);
drawdown_est = zeros(1, N);
crash_detected = false(1, N);

for t = window_size:N
    w = prices(t-window_size+1:t);
    ret = diff(w) ./ w(1:end-1);
    var_est(t) = var(ret);

    peak = max(w);
    drawdown_est(t) = (prices(t) - peak) / peak;

    if var_est(t) > sld_th.Value || drawdown_est(t) < -0.15 || states(t) == 3
        crash_detected(t) = true;
    end
end
```

To conclude, we used hypothesis testing for dictating whether the market is crashing or not. As we can see from the code, if any of the 3 conditions are met, a market is said to crash and will be marked red in the graph of our simulation or dashboard. The 3 conditions for a crash to be detected are:

Conditions for a flagged crash:
1. Variance > Threshold value(0.0015 for this simulation)
2. Drawdown (price drop) by 15% within the time period
3. If the state is already in 3(crash state)

7. Results and Discussion

Table 3 summarises a representative 252-trading-day (one simulated year) run of the completed system. The simulated price begins at $100, rises to a peak of $111.38 on day 22, and then enters a prolonged decline.

Quantity	Value
Peak price (day of peak)	$111.38 (Day 22)
Day crash alert fired	Day 209
10-day rolling variance at alert	0.00175 (threshold: 0.0015)
10-day drawdown at alert	−18.9% (threshold: −15%)
Overall peak-to-trough move	−52.7%
Risk gauge / hypothesis outcome	100% — H_1 accepted

Table 3. Summary of a representative simulated run, as captured from the dashboard.

By day 209, the rolling 10-day sample variance reaches 0.00175 — above the 0.0015 alarm threshold — and the 10-day drawdown reaches −18.9%, beyond the −15% drawdown threshold, so both directly observable conditions are breached at essentially the same time. The overall peak-to-trough decline across the full run reaches −52.7%, and the dashboard's risk gauge correctly pins at 100%, with the system reporting H_1 accepted and a crash alert raised.

This example illustrates two points. First, the two directly observable signals elevated rolling variance and a significant drawdown tend to move together during a genuine regime shift, consistent with the well-documented empirical pattern that market crashes combine falling prices with a simultaneous rise in volatility, rather than either occurring in isolation. Second, because the underlying regime sequence is known in this simulated example, it is possible to confirm that the statistical alarm coincided with, rather than substantially lagged, the onset of the simulated Crash regime.

This single illustrative run demonstrates that the system behaves sensibly on a representative case. It does not, on its own, establish how reliably the detector performs in general; that requires a systematic evaluation across many independent runs, measuring the detector's true detection rate and false-alarm rate, which is identified as a priority improvement in Section 9.

8. Benefits of the Project

Apart from completing the project itself, working on it gave us several useful learning experiences as students.

1. Connecting different course topics: We had to use the concepts of three different topics from Random Signals and Processing course: Markov chains, Brownian-motion, and hypothesis testing and make them work together in one system. This helped us understand how these concepts are connected in a practical situation instead of studying them separately.

2. Applying theory through MATLAB: The project gave us the opportunity to turn the mathematical concepts we learned in class into actual MATLAB code. Working with things like transition matrices,

stochastic equations, test statistics, and significance levels helped us understand the theory more clearly while also improving our simulation and numerical programming skills.

.

3. A useful project for future work: Finally, we now have a working and demonstrable MATLAB application rather than just a theoretical assignment. It can be used as a portfolio project and can also be developed further by adding real market data, more advanced detection methods, or other financial analysis features in the future.

9. Advantages of Our Approach

Compared with a simpler version of the same concepts, our approach has a few clear advantages.

1. More realistic market behaviour: Since the drift and volatility change according to the Markov regime, the simulated stock price does not behave the same way all the time. It can show periods of high and low volatility as well as sudden changes in market conditions, which would be difficult to capture using a basic GBM model with fixed parameters.

2. More meaningful crash detection: Instead of simply saying that a crash has occurred whenever the price drops by a certain percentage, our system uses hypothesis testing to decide whether the observed change is statistically significant. This gives the detection process a proper statistical basis.

3. Interactive simulation: All the main parameters can be changed directly from the dashboard. This makes it easy to see how changing factors such as volatility, drift, or the detection threshold affects the simulation and the crash detector without having to modify the MATLAB code and run it again.

4. Ground truth for testing: Since the simulator generates the actual market regime along with the stock price, we know when a crash regime really occurs. This makes it possible to compare the detector's output with the actual regime and see how accurately it performs. With real historical data, this is much harder because the true underlying regime is not directly labelled.

5. Modular design: The Markov chain, GBM price model, and crash detection system are designed as separate parts. This means we can improve or replace one part without having to completely redesign the others. For example, we could update the transition matrix using historical data or introduce a different crash detection method while keeping the rest of the simulator unchanged.

---

10. Limitations and Recommended Improvements

While the system successfully integrates the three intended concepts into a working, interactive tool, several aspects could be improved in future revisions.

10.1 Detection Threshold Calibration

The current alarm threshold of 0.0015 was selected based on practical observation rather than being calculated from a formal statistical method. A better approach would be to determine the threshold using the chi-square distribution, which is already part of the topics covered in this course. Under the null hypothesis, the statistic involving the sample variance and baseline variance follows a chi-square distribution with degrees of freedom. Therefore, by choosing a significance level such as $\alpha = 0.05$, we can calculate the threshold statistically instead of setting it manually. This would make the crash detection process more objective and reliable.

10.2 Separating Validation from Deployment

The third detection condition, which checks whether the simulated market has entered the Crash regime, is useful for testing and validating the system. It helps us confirm that the two observable indicators, variance and drawdown, are actually detecting the crash around the time when the regime changes. However, this information would not be available in a real-world situation because the actual market regime cannot be directly observed. A better approach would therefore be to keep the regime check only as an internal tool for testing, while the live crash detector should rely only on the two signals that can be obtained from actual market data: variance and drawdown.

10.3 Rigorous Performance Evaluation

The results in Section 6 show how the system performs for one particular simulation run. However, to get a more reliable idea of how well the detector actually works, the simulation should be repeated many times. For each run, the detector's predictions can be compared with the known actual regime to calculate measures such as the probability of detection and probability of false alarm. The results could also be used to create an ROC curve by testing different threshold values. This would provide a more quantitative and reliable evaluation of the detector instead of relying on just one example run.

10.4 Calibration Against Real Data

The transition matrix and regime parameters used in this project are mainly based on assumed values for demonstration purposes rather than being obtained from real market data. A useful improvement would be to estimate these values using historical stock market data. For example, historical trading days could first be classified into different market regimes, and then the frequency of transitions from one regime to another could be calculated. This would make the model more realistic and better connected to actual market behaviour.

10.5 Additional Statistical Depth

Running the simulation many times using a Monte Carlo approach would give a much better understanding of how the model behaves under different conditions. We could then use the Central Limit Theorem to calculate confidence intervals for important results, such as the average maximum drawdown in a year or the typical number of crash days. This would also allow us to use another important concept from the course that was not included in the current version of the project. Instead of looking at just one simulation, we would be able to see the overall range of outcomes the model can produce and make more reliable conclusions about its performance.

10.6 Further Extensions

The project can be further improved by making the simulator more flexible and data-driven. In addition to adding more market regimes, such as Recovery and Correction, the system could allow users to import historical stock market data and analyze it directly. The simulator could use this data to estimate market parameters, identify different market regimes, and build a more realistic transition matrix based on actual market behaviour. These estimated values could then be used to generate simulations that better reflect real-world conditions.

The model could also be extended to simulate multiple correlated assets, allowing us to study crash risk at the portfolio level rather than focusing on a single stock. Another possible extension would be to compare

the current statistical detection method with a machine-learning-based detector and evaluate which approach performs better. Together, these improvements would make the simulator more practical, realistic, and useful for analyzing both historical data and possible future market scenarios.

11. Conclusion

This project aimed to bring together three important concepts from Random Signals and Processes: Markov chains, Geometric Brownian Motion, and statistical hypothesis testing into one practical system. The main goal was to simulate stock prices that could switch between different market conditions and then detect a possible crash while it was happening. The interactive MATLAB dashboard shows that these concepts, although mostly studied from a theoretical perspective, can be combined to create a practical and easy-to-explore application. It also gave us an opportunity to apply the mathematical tools we learned in our EEE coursework to a completely different field.

A sample simulation showed that the system works as expected and responds reasonably to changes in market conditions. However, the project also highlighted a few areas that could be improved in the future. In particular, the detection threshold should be determined using a more rigorous statistical approach instead of being chosen manually. The detector should also be tested over a much larger number of simulations to properly measure its accuracy and reliability. The detector also should be able to import data and analyse it to simulate market behaviors. These improvements would be natural next steps and can be developed using the same concepts and techniques covered in the course.